

FINAL YEAR PROJECT REPORT

Project title:

**Design and development of a multi-agent system for
queue waiting-time estimation from video streams**

Prepared by:

Salmen Ben Ammar

Host Company:



Company Supervisor:

Ramzi Guesmi

Academic Supervisor:

Tarek Hamrouni

Academic Year 2025–2026



Dedications

To my parents,

No dedication could ever truly reflect my unending love and gratitude for all you've done for my education and well-being. May you recognize in this simple work the fulfillment of your dreams and the reward of your many sacrifices.

To my sister,

Your affection, support, and constant encouragement have been an immense source of motivation throughout my journey. Your belief in me helped me overcome challenges and stay focused on my goals. I wish you a life filled with success and happiness.

To my family,

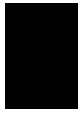
Your presence and unconditional love are my greatest treasures. Thank you for everything. You will forever remain my inexhaustible source of support, love, and comfort.

To all my friends,

My second family who made my journey an indescribable experience, and who have always encouraged me and believed in me. I wish you a future filled with achievement and joy.

To all of you

I dedicate this work.



Acknowledgements

At the conclusion of this project, I wish to express my sincere gratitude to the individuals who have greatly contributed to its successful completion.

First, I would like to express my profound appreciation to the entire team at **Data-DoIt** for graciously providing me the opportunity to carry out this final-year project.

I would also like to extend my sincere thanks to my professional supervisor, **Mr. Ramzi Guesmi**, for his unwavering support, constant availability, patience, and his relevant and constructive guidance, which proved invaluable throughout this project.

I warmly thank my academic supervisor, **Mr. Tarek Hamrouni**, for his constant availability, patience, and insightful advice, which were essential for the successful completion of this work. His guidance, encouragement, and constructive feedback not only helped me overcome numerous challenges but also inspired me to strive for excellence and deepen my understanding throughout this project.

I would also like to take this opportunity to express my gratitude to the entire faculty of **Sesame University**, whose contributions have been fundamental to my education over the past three years.

Finally, I sincerely thank the members of the jury for agreeing to evaluate my work. Their expertise and commitment are greatly appreciated, and I wish to express my deep gratitude for their contribution to the assessment of this project.



Table of Content

List of Figures	vii
List of Tables	viii
General Introduction	ix
1 Context of the Project	1
1.1 Introduction	2
1.2 Host Organization	2
1.2.1 DataDoIt	2
1.2.2 Sectors of Activity	2
1.3 Presentation of the Project	2
1.3.1 Project Context	2
1.3.2 Problem Statement	3
1.3.3 Study of Existing Approaches	3
1.3.3.1 Study of Existing and Similar Solutions	3
1.3.3.2 Limitations of Existing Solutions	3
1.3.4 Proposed Solution	3
1.3.5 Objectives of the Project	3
1.3.6 Target Audience	4
1.4 Project Management Methodology	4
1.4.1 Choice of Methodology: Agile	4
1.4.2 SCRUM	4
1.5 Conclusion	5

2	State of the Art	6
2.1	Computer Vision for Queue Monitoring	7
2.2	People Detection	7
2.3	Multi-Object Tracking	7
2.4	Queueing Modeling	7
2.5	Uncertainty Quantification	7
2.6	Critical Synthesis	8
3	Methodology and Design	9
3.1	Overall Architecture	10
3.2	Processing Pipeline	10
3.3	Indicator Modeling	10
3.3.1	Arrival and Service Rates	10
3.3.2	Waiting Time	11
3.4	Bayesian Uncertainty Quantification	11
3.5	Threshold Detection and Alerts	11
3.6	Technology Choices	11
3.7	Agile Project Management with Scrum	12
3.7.1	Product Backlog	12
3.7.2	Sprint Planning	12
3.7.3	Scrum Ceremonies and Tracking	12
4	Implementation and Development	13
4.1	Overview	15
4.2	Sprint I: Core Detection and Queue Logic	15
4.2.1	Sprint Backlog	15
4.2.2	System Architecture	15

4.2.3	Module Implementation	15
4.2.3.1	Detector Module	15
4.2.3.2	Tracker Module	16
4.2.3.3	Zone Manager	16
4.3	Sprint II: Graphical Interface and Multi-Video Support	16
4.3.1	Sprint Backlog	16
4.3.2	GUI Development	16
4.3.3	Interface Technologies	17
4.3.3.1	Tkinter and CustomTkinter	17
4.3.3.2	Zone Selector Tool	17
4.4	Sprint III: YOLO26 Migration and Performance Tuning	17
4.4.1	Sprint Backlog	17
4.4.2	Model Selection	17
4.4.3	Performance Optimization	17
4.4.3.1	GPU Acceleration	17
4.4.3.2	Inference Settings	17
4.5	Sprint IV: Uncertainty Quantification	18
4.5.1	Sprint Backlog	18
4.5.2	Bayesian Rate Estimation	18
4.5.3	Confidence Intervals	18
4.5.4	Uncertainty Levels	18
4.6	Sprint V: n8n Integration and Alert System	18
4.6.1	Sprint Backlog	18
4.6.2	Logging and Persistence	18
4.6.3	Webhook Integration	19
4.6.3.1	n8n Webhook Client	19
4.6.3.2	Webhook Payload	19

4.6.4	Threshold Detection	19
4.7	Code Organization	19
4.8	Software Validation	20
4.9	Challenges and Solutions	20
5	Results and Evaluation	21
5.1	Experimental Protocol	22
5.2	Observed Metrics	22
5.3	Qualitative Analysis	22
5.4	Example Results Table	22
5.5	Evaluation Limitations	23
6	Discussion	24
6.1	Project Contributions	25
6.2	Limitations	25
6.3	Comparison with Field Requirements	25
7	Conclusion and Future Work	26
7.1	General Conclusion	27
7.2	Objective Review	27
7.3	Future Work	27
A	Appendices	28
A.1	Useful Commands	28
A.2	Example Webhook Payload	28
A.3	Items to Add Before Final Submission	28



List of Figures



List of Tables

5.1	Example experimental summary (to be completed)	22
-----	--	----



General Introduction

In a context where digital transformation and artificial intelligence increasingly shape operational decision-making, service organizations are under strong pressure to reduce waiting time and improve service quality. Supermarkets, banks, and service centers face the same recurring challenge: queues are dynamic, difficult to monitor manually, and highly sensitive to demand fluctuations.

In practice, customers often join a queue without a clear idea of expected waiting time, while managers and operational staff do not always have reliable real-time indicators to adapt resources quickly. Manual observation remains subjective and costly, while simple counting solutions do not provide robust waiting-time estimation with confidence levels.

This end-of-studies project addresses this gap through the design and development of an intelligent real-time queue monitoring system based on computer vision and queueing theory, with explicit uncertainty quantification. The proposed system detects and tracks people from video streams, analyzes occupancy inside configurable queue zones, estimates arrival and service rates, computes waiting time, and exposes confidence-aware metrics for operational use.

This report is organized as follows. Chapter 1 presents the context of the project, host organization, problem statement, and objectives. Chapter 2 introduces the state of the art. Chapter 3 details the methodology and system design. Chapter 4 presents implementation and development through sprint-based delivery. Chapter 5 reports results and evaluation. Chapter 6 discusses contributions and limitations. Chapter 7 concludes the work and outlines future directions.

Context of the Project

Table of Content

1.1	Introduction	2
1.2	Host Organization	2
1.2.1	DataDoIt	2
1.2.2	Sectors of Activity	2
1.3	Presentation of the Project	2
1.3.1	Project Context	2
1.3.2	Problem Statement	3
1.3.3	Study of Existing Approaches	3
1.3.4	Proposed Solution	3
1.3.5	Objectives of the Project	3
1.3.6	Target Audience	4
1.4	Project Management Methodology	4
1.4.1	Choice of Methodology: Agile	4
1.4.2	SCRUM	4
1.5	Conclusion	5

1.1 Introduction

In this chapter, we present the general context of the project. We begin with the host organization and its sectors of activity. Then, we introduce the project presentation, including the problem statement, a brief study of existing approaches, and the proposed solution. Finally, we describe the methodology adopted to guide the work throughout the project.

1.2 Host Organization

1.2.1 DataDoIt

This final-year project was carried out in collaboration with DataDoIt, which provided the professional framework, guidance, and practical use-case context.

1.2.2 Sectors of Activity

The problem addressed by this work is relevant to several sectors where queue congestion impacts service quality, including retail checkout systems, banking agencies, and customer service centers.

1.3 Presentation of the Project

1.3.1 Project Context

This project focuses on real-time queue monitoring from video streams in order to transform raw visual data into operational indicators for decision support.

1.3.2 Problem Statement

Manual approaches or simple counting-based methods remain limited:

- they are time-consuming,
- they do not explicitly estimate waiting time,
- they do not report uncertainty in the measurements.

1.3.3 Study of Existing Approaches

1.3.3.1 Study of Existing and Similar Solutions

Existing solutions are generally based on manual counting, basic counters, or isolated analytics components that do not provide a full end-to-end queue estimation pipeline.

1.3.3.2 Limitations of Existing Solutions

Most existing approaches either ignore uncertainty, provide delayed analytics, or fail to combine detection, tracking, queue modeling, and actionable alerting in one integrated system.

1.3.4 Proposed Solution

The proposed system combines person detection, multi-object tracking, zone-based filtering, queue-theory metrics, and Bayesian uncertainty quantification with dashboard and webhook integration.

1.3.5 Objectives of the Project

This work aims to design a system able to:

1. detect and track people in real time,

2. count customers inside a configurable queue zone,
3. estimate arrival rate λ and service rate μ ,
4. compute expected waiting time W ,
5. quantify uncertainty for key metrics,
6. provide a monitoring interface and external integration (n8n).

1.3.6 Target Audience

The main audience includes operational staff (for real-time supervision), managers (for resource planning), and end-users who benefit from reduced waiting time and improved service flow.

1.4 Project Management Methodology

1.4.1 Choice of Methodology: Agile

The project follows an incremental and iterative approach to deliver functional modules progressively while integrating feedback and resolving technical risks early.

1.4.2 SCRUM

Development was organized in explicit Scrum sprints with incremental deliverables: Sprint 1 (core perception pipeline), Sprint 2 (GUI and multi-video support), Sprint 3 (YOLO26 migration and performance tuning), Sprint 4 (uncertainty quantification), and Sprint 5 (n8n integration and alerting).

1.5 Conclusion

This chapter introduced the project context, organizational setting, problem statement, and proposed approach. The next chapter presents the state of the art supporting the technical choices made in this work.

Table of Content

2.1	Computer Vision for Queue Monitoring	7
2.2	People Detection	7
2.3	Multi-Object Tracking	7
2.4	Queueing Modeling	7
2.5	Uncertainty Quantification	7
2.6	Critical Synthesis	8

2.1 Computer Vision for Queue Monitoring

Object detection and tracking in dynamic scenes are fundamental building blocks for intelligent monitoring. Modern approaches mainly rely on convolutional-neural-network detectors and multi-object tracking algorithms.

2.2 People Detection

The YOLO family offers a strong trade-off between accuracy and latency for real-time applications. In this project, YOLO26 variants are selected based on operational needs (speed vs. precision).

2.3 Multi-Object Tracking

Tracking keeps a consistent identity for each person across consecutive frames. ByteTrack is chosen for its robustness and straightforward integration into a real-time pipeline.

2.4 Queueing Modeling

Queueing theory provides a mathematical framework to estimate service-system performance. The M/M/1 model is selected as a first approximation, under the stability condition $\lambda < \mu$.

2.5 Uncertainty Quantification

Point estimates alone can be misleading in noisy environments. Bayesian approaches, especially Gamma distributions for Poisson rates, make it possible to produce inter-

pretable credible intervals.

2.6 Critical Synthesis

Existing work often focuses on detection or counting, with less emphasis on:

- explicit waiting-time estimation,
- uncertainty quantification,
- operational integration (alerts, dashboard, webhook).

This final year project is positioned precisely at this intersection.

Table of Content

3.1	Overall Architecture	10
3.2	Processing Pipeline	10
3.3	Indicator Modeling	10
3.3.1	Arrival and Service Rates	10
3.3.2	Waiting Time	11
3.4	Bayesian Uncertainty Quantification	11
3.5	Threshold Detection and Alerts	11
3.6	Technology Choices	11
3.7	Agile Project Management with Scrum	12
3.7.1	Product Backlog	12
3.7.2	Sprint Planning	12
3.7.3	Scrum Ceremonies and Tracking	12

3.1 Overall Architecture

The system is structured in layers:

1. video acquisition,
2. perception (detection + tracking + zone filtering),
3. analysis (metrics + uncertainty + thresholds),
4. output (visualization, CSV, webhook).

3.2 Processing Pipeline

For each frame:

- people detection,
- temporal association of trajectories,
- polygon-zone membership verification,
- update of entry/exit events,
- computation of queue indicators.

3.3 Indicator Modeling

3.3.1 Arrival and Service Rates

Observed events over a time window are used to estimate:

$$\lambda = \frac{N_{\text{arrivals}}}{\Delta t}, \quad \mu = \frac{N_{\text{exits}}}{\Delta t}. \quad (3.1)$$

3.3.2 Waiting Time

Under the M/M/1 assumption, one form used in the implementation is:

$$W = \frac{\lambda}{\mu - \lambda}, \quad \text{with } \lambda < \mu. \quad (3.2)$$

3.4 Bayesian Uncertainty Quantification

Rates are modeled using posterior Gamma distributions:

$$\lambda \sim \Gamma(\alpha_\lambda, \beta_\lambda), \quad \mu \sim \Gamma(\alpha_\mu, \beta_\mu). \quad (3.3)$$

Ninety-five percent credible intervals are computed for each metric.

3.5 Threshold Detection and Alerts

A dedicated module flags critical situations:

- excessive waiting time,
- high queue backlog,
- arrival spikes,
- service degradation,
- instability or high uncertainty.

3.6 Technology Choices

- Python, OpenCV, NumPy, Pandas,
- YOLO26 (Ultralytics) for detection,

- Supervision + ByteTrack for tracking,
- SciPy for statistical computation,
- Tkinter/CustomTkinter for the interface,
- Requests + n8n for external integration.

3.7 Agile Project Management with Scrum

3.7.1 Product Backlog

The product backlog was organized around business value and technical risk reduction. Priority user stories targeted: real-time queue visibility, reliable wait-time estimation, uncertainty-aware decision support, and external alerting through n8n/Telegram.

3.7.2 Sprint Planning

Planning was performed in short iterations, each with a clear sprint goal, implementation scope, and validation criteria. Work was decomposed into incremental deliverables to ensure traceable progress from detection pipeline to full integration.

3.7.3 Scrum Ceremonies and Tracking

Progress tracking relied on iterative review of completed features, pending tasks, and blockers. Sprint outcomes were documented in project logs, and backlog priorities were updated according to testing feedback and integration constraints.

Table of Content

4.1	Overview	15
4.2	Sprint I: Core Detection and Queue Logic	15
4.2.1	Sprint Backlog	15
4.2.2	System Architecture	15
4.2.3	Module Implementation	15
4.3	Sprint II: Graphical Interface and Multi-Video Support	16
4.3.1	Sprint Backlog	16
4.3.2	GUI Development	16
4.3.3	Interface Technologies	17
4.4	Sprint III: YOLO26 Migration and Performance Tuning	17
4.4.1	Sprint Backlog	17
4.4.2	Model Selection	17
4.4.3	Performance Optimization	17
4.5	Sprint IV: Uncertainty Quantification	18
4.5.1	Sprint Backlog	18
4.5.2	Bayesian Rate Estimation	18
4.5.3	Confidence Intervals	18
4.5.4	Uncertainty Levels	18
4.6	Sprint V: n8n Integration and Alert System	18
4.6.1	Sprint Backlog	18
4.6.2	Logging and Persistence	18

4.6.3	Webhook Integration	19
4.6.4	Threshold Detection	19
4.7	Code Organization	19
4.8	Software Validation	20
4.9	Challenges and Solutions	20

4.1 Overview

The project is structured in five Scrum sprints, each delivering an operational increment of the final system, from core computer vision to uncertainty-aware monitoring and external integration.

4.2 Sprint I: Core Detection and Queue Logic

4.2.1 Sprint Backlog

Main backlog items for this sprint were person detection, multi-object tracking, and queue-zone filtering.

4.2.2 System Architecture

The foundation layer consists of three core components:

- YOLO26 person detection,
- ByteTrack object tracking,
- Polygon zone filtering.

4.2.3 Module Implementation

4.2.3.1 Detector Module

The detector module wraps Ultralytics YOLO for real-time person detection:

- Supports model sizes: nano, small, medium, large, xlarge,
- Returns bounding boxes with confidence scores.

4.2.3.2 Tracker Module

ByteTrack assigns consistent IDs across frames to prevent duplicate counting.

4.2.3.3 Zone Manager

Polygon-based filtering ensures only people inside the configured queue zone are analyzed.

4.3 Sprint II: Graphical Interface and Multi-Video Support

4.3.1 Sprint Backlog

Main backlog items for this sprint were user workflow design, per-video zone configuration, and multi-source execution.

4.3.2 GUI Development

The interface follows a simple workflow:

1. choose the number of videos,
2. select sources,
3. configure the model,
4. define one zone per video,
5. launch parallel analysis.

4.3.3 Interface Technologies

4.3.3.1 Tkinter and CustomTkinter

Primary framework for the graphical interface with custom styling.

4.3.3.2 Zone Selector Tool

Interactive polygon selection for queue area definition per video source.

4.4 Sprint III: YOLO26 Migration and Performance Tuning

4.4.1 Sprint Backlog

Main backlog items for this sprint were model migration, inference optimization, and runtime stability improvements.

4.4.2 Model Selection

Migration from earlier YOLO versions to YOLO26 nano/small for optimal speed-accuracy trade-off.

4.4.3 Performance Optimization

4.4.3.1 GPU Acceleration

PyTorch CUDA support for real-time inference.

4.4.3.2 Inference Settings

Tuning confidence thresholds and NMS parameters for operational requirements.

4.5 Sprint IV: Uncertainty Quantification

4.5.1 Sprint Backlog

Main backlog items for this sprint were Bayesian rate modeling, confidence intervals, and uncertainty-level classification.

4.5.2 Bayesian Rate Estimation

Rates are modeled using posterior Gamma distributions:

$$\lambda \sim \Gamma(\alpha_\lambda, \beta_\lambda), \quad \mu \sim \Gamma(\alpha_\mu, \beta_\mu). \quad (4.1)$$

4.5.3 Confidence Intervals

Ninety-five percent credible intervals are computed for each metric.

4.5.4 Uncertainty Levels

Classification of uncertainty as Low, Medium, or High based on posterior variance.

4.6 Sprint V: n8n Integration and Alert System

4.6.1 Sprint Backlog

Main backlog items for this sprint were webhook integration, payload standardization, alert threshold detection, and resilient data persistence.

4.6.2 Logging and Persistence

Metrics are persisted locally in CSV format for reliability and traceability.

4.6.3 Webhook Integration

4.6.3.1 n8n Webhook Client

HTTP POST requests send JSON metrics every 5 seconds with retry logic.

4.6.3.2 Webhook Payload

The payload includes 17 metrics: timestamp, queue count, rates, waiting time, and uncertainty level.

4.6.4 Threshold Detection

A dedicated module flags critical situations:

- excessive waiting time,
- high queue backlog,
- arrival spikes,
- service degradation,
- instability or high uncertainty.

4.7 Code Organization

The project is organized into specialized modules:

- `detector.py`: people detection,
- `tracker.py`: object tracking,
- `zone_manager.py`: polygon-zone management,
- `queue_analyzer.py`: queue metric computation,
- `uncertainty.py`: Bayesian uncertainty estimation,

- `threshold_detector.py`: critical condition detection,
- `gui/app.py`: graphical interface and multi-video launcher.

4.8 Software Validation

Unit tests are included (notably for the interface and uncertainty modules). Validation also covered real-time runtime scenarios on video streams.

4.9 Challenges and Solutions

This project required several engineering adjustments:

- stabilization of GUI rendering,
- improved robustness of zone selection,
- calibration of the performance/accuracy trade-off,
- handling edge cases in the queueing model.

Table of Content

5.1	Experimental Protocol	22
5.2	Observed Metrics	22
5.3	Qualitative Analysis	22
5.4	Example Results Table	22
5.5	Evaluation Limitations	23

5.1 Experimental Protocol

Experiments were conducted on queue-scene videos with different traffic dynamics. The monitored indicators include counting accuracy, estimation stability, and alert responsiveness.

5.2 Observed Metrics

- number of people inside the zone,
- arrival rate λ ,
- service rate μ ,
- estimated waiting time W ,
- credible intervals and uncertainty level.

5.3 Qualitative Analysis

The system correctly tracks queue-load evolution and flags critical phases. The addition of credible intervals improves the operational interpretation of results.

5.4 Example Results Table

Table 5.1: Example experimental summary (to be completed)

Scenario	λ (pers/s)	μ (pers/s)	W (s)	Uncertainty Level
Low load	0.04	0.08	1.00	Low
Medium load	0.08	0.11	2.67	Medium
High load	0.10	0.11	10.00	High

5.5 Evaluation Limitations

Comprehensive multi-scenario evaluation still needs to be extended (camera-angle variations, strong occlusions, lighting changes, multi-zone setups).

Table of Content

6.1	Project Contributions	25
6.2	Limitations	25
6.3	Comparison with Field Requirements	25

6.1 Project Contributions

The proposed system demonstrates that computer vision and queueing theory can be combined to produce actionable real-time indicators. Uncertainty quantification is a major contribution because it helps avoid decisions based on overconfident estimates.

6.2 Limitations

- simplifying assumptions of the M/M/1 model,
- sensitivity to detection quality in complex scenes,
- dependence on proper zone configuration,
- current lack of native multi-zone support in the analytics core.

6.3 Comparison with Field Requirements

The prototype meets real-time monitoring and decision-support needs. For large-scale operational deployment, an additional industrialization and extensive evaluation phase is still required.

Table of Content

7.1	General Conclusion	27
7.2	Objective Review	27
7.3	Future Work	27

7.1 General Conclusion

This final year project enabled the design and development of a complete waiting-time estimation system from video streams. The implemented architecture covers the full chain, from visual perception to business-oriented metrics, with integrated uncertainty quantification.

7.2 Objective Review

The main objectives are achieved:

- people detection/tracking,
- queue-metric estimation,
- real-time display,
- logging and n8n integration,
- explicit uncertainty handling.

7.3 Future Work

Potential extensions include:

- multi-zone support,
- advanced Telegram alerting through n8n,
- richer Streamlit dashboard,
- edge optimization (ONNX / TensorRT),
- broader statistical evaluation protocol.



A.1 Useful Commands

- Run CLI analysis: `python -m src.main -help`
- Launch the interface: `python gui.py`

A.2 Example Webhook Payload

```
{  
  "timestamp": "2026-02-21T11:02:30",  
  "queue_count": 7,  
  "arrival_rate": 0.09,  
  "service_rate": 0.11,  
  "wait_time_sec": 4.5,  
  "uncertainty_level": "Medium"  
}
```

A.3 Items to Add Before Final Submission

- official logos (university/company),
- dedication and acknowledgments page (if required),
- list of acronyms,
- final architecture figures,

- final quantitative results and detailed discussion.



Abstract

This final year project addresses real-time waiting-time estimation in checkout queues from video streams. The proposed system combines object detection (YOLO), multi-object tracking (ByteTrack), polygon-based zone filtering, and an M/M/1 queuing model to compute key metrics (λ , μ , W). To improve decision reliability, Bayesian uncertainty quantification is integrated through 95% credible intervals.

The implementation demonstrates the feasibility of automated queue monitoring with a real-time dashboard, CSV logging, and webhook integration with n8n. The approach supports operational decision-making while explicitly reporting confidence levels of estimated metrics.

Keywords: Computer vision, YOLO, ByteTrack, queueing theory, M/M/1, Bayesian uncertainty, real-time monitoring.